

Clase 009 — PowerShell para seguridad ofensiva y defensiva

Parte: 0 — Fundamentos y prerequisites · Fuente: Microsoft PowerShell Documentation 🕒 Duración estimada: 110 min · Nivel: Fundamentos

Objetivo

Aprender PowerShell como herramienta dual: los atacantes lo usan para vivir de la tierra (*living off the land*) y los defensores para automatizar y responder. Al terminar sabrás manejar objetos, consultar el sistema, entender por qué PowerShell es tan potente para ataque y qué controles (logging, AMSI, políticas) lo vigilan.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Usar** cmdlets, la tubería de objetos y el sistema de ayuda.
- 2. **Consultar** procesos, servicios, red y eventos con PowerShell.
- 3. **Explicar** por qué PowerShell es un vector ofensivo frecuente.
- 4. **Configurar** defensas: logging de scripts, políticas de ejecución, AMSI.
- 5. **Escribir** un script de recolección para respuesta a incidentes.

Temas

#	Tema	Por qué importa
1	Cmdlets y verbo-nombre	Sintaxis consistente y descubrible
2	Pipeline de objetos	A diferencia de Bash, fluyen objetos, no texto
3	<code>Get-Help</code> / <code>Get-Member</code>	Autodescubrimiento del entorno
4	Consulta del sistema	Procesos, servicios, red, eventos
5	Uso ofensivo	Descarga y ejecución, LOLBins
6	Execution Policy	Qué es y por qué no es seguridad
7	Logging y AMSI	Script Block Logging, transcripción, antimalware
8	Constrained Language Mode	Mitigación de ejecución arbitraria

Definiciones y características

- **Cmdlet**: comando nativo con forma `Verbo-Nombre` (`Get-Process`). Clave: devuelve objetos .NET, no texto.
- **Pipeline de objetos**: encadena cmdlets pasando objetos con propiedades. Clave: `Where-Object` , `Select-Object` , `Sort-Object` .

- **Execution Policy:** ajuste que controla qué scripts se ejecutan. Clave: **no** es una barrera de seguridad; se evita fácilmente.
- **AMSI (Antimalware Scan Interface):** interfaz que permite al antivirus inspeccionar scripts en memoria. Clave: los atacantes intentan evadirla.
- **Script Block Logging:** registro del contenido de los bloques ejecutados (evento 4104). Clave: pilar de la detección de abuso de PowerShell.
- **Constrained Language Mode:** modo que limita el acceso a APIs peligrosas. Clave: reduce la capacidad ofensiva de PowerShell.

Herramientas y preparación

Trabaja en tu VM Windows de laboratorio. PowerShell viene integrado; usa la consola y el **ISE** o **VS Code** con la extensión de PowerShell. Para defensa, explora la Directiva de grupo local (`gpedit.msc`) y el Visor de eventos (`Microsoft-Windows-PowerShell/Operational`). No ejecutes cargas ofensivas contra sistemas ajenos.

Laboratorio guiado

1. Descubrir el entorno:

```
powershell Get-Command -Verb Get | Measure-Object Get-Help Get-Process -Examples Get-Process | Get-Member
```

2. Consultas de sistema (uso defensivo/forense):

```
powershell Get-Process | Sort-Object CPU -Descending | Select-Object -First 5 Name,Id,CPU Get-Service | Where-Object Status -eq 'Running' | Select-Object Name,DisplayName Get-NetTCPConnection | Where-Object State -eq 'Listen'
```

3. Eventos de seguridad. Últimos fallos de inicio de sesión:

```
powershell Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 5
```

4. Execution Policy. Compruébala y observa que no impide todo:

```
powershell Get-ExecutionPolicy -List
```

Reflexiona: `-ExecutionPolicy Bypass` la anula sin permisos especiales. 5. **Activar defensas.** Habilita Script Block Logging por directiva local (Configuración del equipo → Plantillas administrativas → Componentes de Windows → Windows PowerShell). Genera un script y verifica el evento **4104**. 6. **Script de recolección IR.** Crea `recolectar.ps1` que exporte a CSV procesos, servicios en ejecución y conexiones de red con marca de tiempo.

 **Nota ética:** las capacidades ofensivas de PowerShell (descarga y ejecución en memoria, evasión) se estudian aquí **solo** para entender la detección y **exclusivamente** en tu laboratorio. Usarlas contra terceros sin autorización es un delito.

Ejercicios

1. Escribe un one-liner que liste los 10 procesos con más memoria (WorkingSet) ordenados.
2. Exporta a CSV todos los servicios con inicio automático que están detenidos.
3. Explica por qué `Get-ExecutionPolicy` no protege realmente contra scripts maliciosos.

4. Investiga qué es un LOLBin y da dos ejemplos usados vía PowerShell.
5. Configura y verifica la transcripción de PowerShell (PowerShell Transcription).
6. Detecta con `Get-WinEvent` cualquier evento 4104 que contenga la palabra `Invoke-Expression`.

Reto verificable

Construye `triage.ps1`, un script de recolección para respuesta a incidentes que genere un informe (CSV o JSON) con: procesos y sus rutas, servicios en ejecución, conexiones de red en escucha, usuarios locales y entradas de auto-inicio. Habilita Script Block Logging y demuestra que la ejecución del script queda registrada en el evento 4104.

Criterio de aceptación: el script produce un informe estructurado sin errores en la VM, y en el registro `Microsoft-Windows-PowerShell/Operational` aparece un evento 4104 correspondiente a su ejecución. Otro alumno puede correr el script y obtener un informe equivalente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"running scripts is disabled on this system"	Execution Policy Restricted. Ajusta con <code>Set-ExecutionPolicy</code> (con permiso) o usa <code>-File</code> con política adecuada.
El pipeline no filtra como esperas	Estás tratando objetos como texto. Usa <code>Get-Member</code> para ver propiedades reales y <code>Where-Object</code> .
No aparecen eventos 4104	Script Block Logging no habilitado. Actívalo por directiva y reintenta.
<code>Get-WinEvent</code> lento o sin resultados	Filtro incorrecto. Usa <code>-FilterHashtable</code> en vez de filtrar tras traer todo.
El script funciona en ISE pero no en consola	Diferencias de contexto/perfil. Prueba siempre en la consola destino.

Preguntas frecuentes

? ¿Por qué a los atacantes les gusta PowerShell? Está preinstalado, firmado por Microsoft, accede a .NET/WMI/APIs y puede ejecutar código en memoria sin tocar disco. Es *living off the land* por excelencia.

? ¿Execution Policy protege mi equipo? No de un atacante decidido: se puede evadir con un parámetro. Es una salvaguarda contra ejecución accidental, no un control de seguridad.

? ¿Qué versión de PowerShell debo aprender? Los conceptos son iguales en Windows PowerShell 5.1 y en PowerShell 7+. Para defensa, 5.1 sigue siendo omnipresente en equipos corporativos.

? ¿AMSI detiene todo? No, hay técnicas de evasión, pero combinado con Script Block Logging y Constrained Language Mode eleva mucho la barrera para el atacante.

Referencias

- Microsoft PowerShell Documentation — <https://learn.microsoft.com/powershell/>
- About Execution Policies — https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies
- AMSI overview — <https://learn.microsoft.com/windows/win32/amsi/antimalware-scan-interface-portal>

- LOLBAS Project — <https://lolbas-project.github.io/>

Siguiente clase

Clase 010 - Redes TCP/IP: modelo OSI, encapsulacion y capas